

1. Candidate should not be referred twice to the same company

Idea

The system should prevent storing duplicate referrals for the same candidate and company.

Explanation

When a user refers a candidate to a company, we store the referral in the database. Each referral will usually contain fields like `userId`, `candidateId`, and `companyId`.

To make sure the same candidate is not referred twice to the same company, we can enforce a rule at the database level by creating a **unique constraint** on the combination of `candidateId` and `companyId`. This means the database will reject any attempt to insert another referral with the same candidate and company.

We can also add a check in the backend service. Before saving a referral, the application can query the database to see if a referral already exists.

Example approach

In Spring Boot, the repository could have a method like:

```
existsByCandidateIdAndCompanyId()
```

If it returns true, we return an error saying the candidate has already been referred to that company.

This ensures the rule is enforced both in the application and the database.

2. What happens when you type a URL in the browser

When a user types a URL into the browser, several steps happen before the webpage is displayed.

First, the browser checks its **cache** to see if it already has a stored version of the page. If it is not found, the browser performs a **DNS lookup** to convert the domain name (for example google.com) into an IP address.

Once the IP address is found, the browser sends an **HTTP or HTTPS request** to the server at that address.

The server receives the request, processes it, and sends back a **response**. The response usually contains HTML, CSS, JavaScript, and other resources needed to display the webpage.

Finally, the browser parses the HTML, builds the **DOM**, loads additional resources like CSS and JavaScript, and renders the page on the screen.

3. Java and Spring Boot version

I currently use **Java 17** and **Spring Boot 3.x** in my projects. Java 17 is a long-term support version and works well with modern Spring Boot versions. Spring Boot helps simplify backend development by providing auto-configuration, dependency management, and easy setup for web applications.

4. Explanation of annotations

@SpringBootApplication

This annotation is used on the main class of a Spring Boot application. It combines multiple configurations such as auto-configuration, component scanning, and configuration setup so that the application can start with minimal configuration.

@Entity

The **@Entity** annotation marks a class as a JPA entity. It means the class will be mapped to a table in the database and each object represents a row in that table.

@Service

The **@Service** annotation is used in the service layer. Classes marked with this annotation usually contain business logic and are managed by the Spring container.

@Autowired

`@Autowired` is used for dependency injection. It tells Spring to automatically provide the required object instead of manually creating it.

@Repository

The `@Repository` annotation is used for classes that interact with the database. It also provides exception translation for database operations.

@Id

The `@Id` annotation marks a field as the primary key of an entity. This field uniquely identifies each record in the table.

@Transactional

`@Transactional` ensures that a set of database operations are executed as a single transaction. If an error occurs during the process, all changes are rolled back to maintain data consistency.

5. GitHub repository

Here you should provide a link to a repository that demonstrates your work with Java and Spring Boot. The repository should ideally include REST APIs, database interaction using JPA or Hibernate, and a clear project structure.

Example format:

GitHub Repository:

<https://github.com/yourusername/project-name>